Break Before You Shape

Exact line breaking from a precompiled, certified width model

Draft · 2026-11-03 · every number in this paper is read from result files produced on one Apple M3 machine

Abstract. Text engines shape a paragraph before they can break it into lines, because widths depend on kerning, ligatures and contextual forms. We show that, for the fonts and scripts we test, a line's shaped width is a *local* function of its text, and that this locality can be *proven* from a font's layout tables. We compile a shaper's behaviour into a small per-font model by measuring it as a black box, and a static analysis of OpenType and AAT tables certifies the lines on which the model is exact. Lines are then broken before anything is shaped. Across 449 local font faces, 4,812,111 certified lines contain no error. Inside Flutter, a 10 KB (gzipped) file built once on the developer's machine reproduces Flutter's own line breaks and paragraph heights in 2,524,902 of 2,524,902 paragraph layouts (nine corpora in six languages written in Latin, Greek and Cyrillic, six font sizes, twelve widths), and in every layout checked on an iOS simulator (192,193) and an Android emulator (192,176). It hands 0.87% of layouts back to the engine and computes paragraph heights 18–25× faster than `TextPainter`.

1 · Introduction

Every mainstream text stack (Chrome, WebKit, Android, Skia's SkParagraph, CoreText, DirectWrite) runs the same order: *shape* the paragraph with a shaper such as HarfBuzz, *break* it into lines using the shaped advances, and often *reshape* around the breaks. Shaping is the expensive step, and much of it is wasted. A chat list computing thousands of item heights, or an editor re-wrapping on every resize, shapes text that it never draws. Figure 1 shows the alternative this paper makes exact: break first, from a precompiled model, then shape only what is drawn.

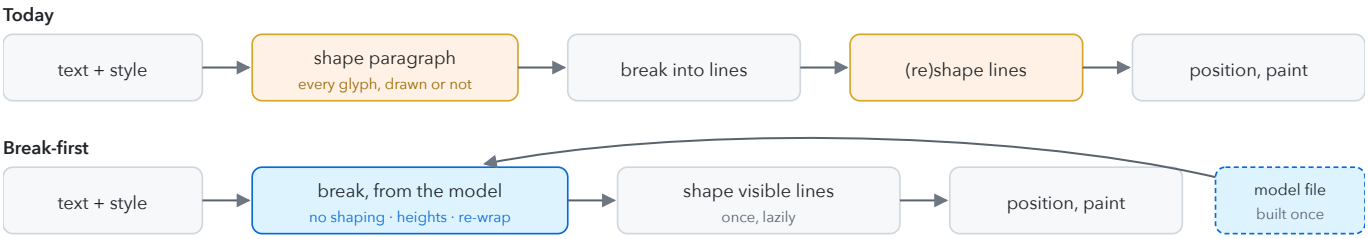


Figure 1. Shape-first layout shapes every glyph before it knows where lines end. Break-first computes line breaks and heights from a precompiled model, so text that is measured but never drawn is never shaped.

Contributions.

1. **Distillation.** A shaper is measured as a black box (every character, every pair, every ligature island in context) and compiled into exact tables. The model matches the target engine by construction.
2. **A certificate.** A static analysis of OpenType (GSUB, GPOS, GDEF) and AAT (`morx`, `kern`, `kerx`) tables proves on which lines the model's width equals the shaped width.
3. **Break-first layout.** Width-independent records give every candidate line's width in O(1), so a resize costs one greedy pass. Where the host engine's behaviour is uncertain, the breaker defers instead of guessing.
4. **Evidence.** A census of 449 font faces, and a Flutter implementation checked against Flutter's own layout on three platforms.

2 · Width is local, and can be distilled

A line's shaped width is the sum of its glyph advances plus positioning adjustments. In practice those adjustments are *local*: pair kerning touches two adjacent glyphs, and a ligature replaces a short run of letters. So the width of any line decomposes into a few kinds of terms: one advance per character, one term per adjacent pair, and a term for each ligature *island* with its two neighbours. Figure 2 shows a real decomposition, measured from Flutter.

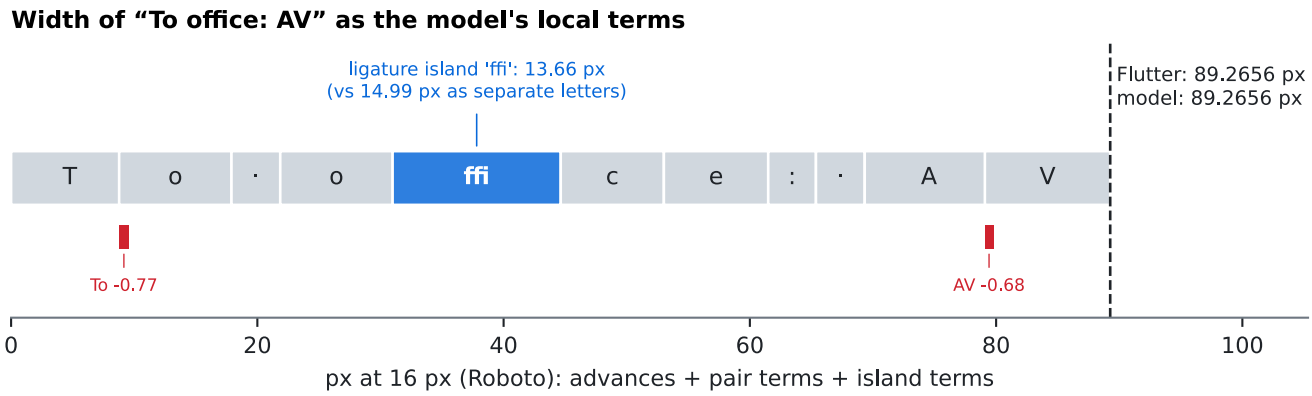


Figure 2. “To office: AV” in Roboto at 16 px. Gray: advances alone, which sum to 92.055 px. Red: pair terms (kerning). Blue: a ligature island. The model sums to 89.2656 px; Flutter measures 89.2656 px.

Distillation. We do not parse kerning or ligature tables. We ask the target shaper. For Flutter, the oracle is Flutter's own `ui.Paragraph`, run under `flutter_tester` on the developer's machine:

- every character alone;
- every ordered pair;
- every certificate island alone, and next to every safe neighbour (§3).

We measure at a font size equal to the font's units per em, so values are font units. All 614,675 measurements came back as exact integers, and widths scale linearly with size (§5.5), so **one file serves every size**. For Roboto (Latin, Greek and Cyrillic, 699 characters), identical rows and columns of the pair matrix collapse into 114×121 classes. With 90 islands, the file is 560 KB, 10.1 KB gzipped, and builds in 8.5 s.

3 · A certificate from the font's tables

Locality is not guaranteed: a contextual substitution can make a glyph depend on text arbitrarily far away. So each character is classified from the font's layout tables, without running the shaper:

- **S** (safe): no substitution can change or consume its glyph;
- **P** (primer): it can start a substitution, such as the first letter of a ligature;
- **K** (passive): it can only be consumed by a substitution a primer started;
- **U** (unsafe): everything else.

An *island* is a maximal run of P/K characters that contains a primer.

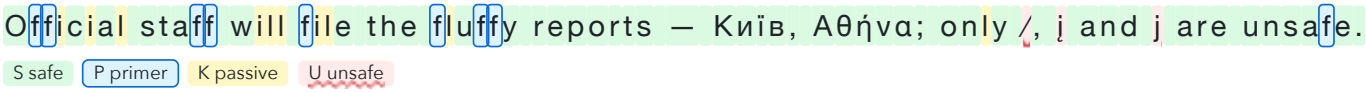


Figure 3. Roboto's certificate on sample text. Islands form only around *f*. Unsafe characters send their paragraph to the engine.

> **Theorem.** Let a line contain no U character, and let every island in it have at most m characters. Then the shaped width of the line equals the model's width.

Proof sketch.

1. **Substitution leaves S glyphs untouched, and islands are closed.**
   - *OpenType.* A single, multiple or alternate lookup fires only at its coverage glyphs, which are U. A contextual rule fires only if every position matches a glyph that can occur, so for each live rule the cheapest position's characters are made U. Ligatures start at P and consume only P/K glyphs.
   - *AAT.* Action reachability is computed over the alphabet; subtables that cannot act are the identity. In the rest, S classes return every reachable state to a start state with no substitution, the start states agree, end of text has no effect after an island, and a bounded exhaustive check shows no action inside an island reaches state stored before it.
2. **Positioning is pair-additive.** Only single adjustments, PairPos with ValueFormat2 = 0, and `kern` or `kerx` pair formats can fire.
3. **Width splits into local terms.** By 1 and 2, the width is a sum of terms, each touching at most one island and its S neighbours.
4. **The terms agree.** The model's terms are measured values of exactly those terms. Islands are measured with every S neighbour, and the checker verifies no model term spans two islands. ∎

The proof trusts that the shaper implements OpenType and AAT as specified, that fontTools parses the tables correctly, and the analyzer's code, which is tested but not machine-checked. By default it also trusts that the shaper selects lookups as HarfBuzz does (script, language, default variation coordinates); a strict mode drops that assumption.

4 · Breaking before shaping, the way the host does

For each paragraph, one pass builds a width prefix in units, and a second finds break opportunities. Both are independent of the box width, so re-wrapping at a new width only re-runs the greedy pass over these records. To reproduce a specific engine, the breaker must also reproduce that engine's conventions. We measured Flutter's rather than assuming them (Table 1), and the breaker **defers** whenever a decision depends on something it cannot certify.

Flutter (SkParagraph) convention	How it was established	Breaker
Shapes once, breaks on shaped cluster advances; kerning sits on the left glyph	Exactness against Flutter's breaks	Pair terms attach to the left character
Line fits if its width, floored to 1/100 px in float32, ≤ box	Accepted and rejected widths around the box edge	Same test, emulated in float32
Line height = font line height, rounded to whole px	Every height in Table 2 (2,524,902 on macOS)	lines × rounded height
No kerning across script runs; spaces join the preceding run	"Time Machine" inside Russian text	Pair term 0 at a script change
UAX #14 with Unicode 15.1 number rules; word-initial dash binds	Per-class tallies of taken vs refused breaks	Rule table for the characters it handles
Sometimes breaks at a space inside an unbreakable segment	Examples (e.g. `self ⏎!= 1.`)	Defers when such a space could end the line
Positions drift in float32 at some sizes	Drift by size (§5.5)	Defers decisions the drift could flip
Android truncates the font size to 1/64 px (e.g. 18.4 → 18.390625); macOS and iOS do not	Effective size implied by measured widths at 8 sizes, on the emulator	One probe layout at start-up detects it; sizes are truncated the same way

Table 1. Host conventions the breaker reproduces, each measured from Flutter.

5 · Evaluation

5.1 · Census: how often does the certificate apply?

We ran the analyzer and the checker on every font face on the machine that covers basic Latin (449 faces). For each, the checker compares every certified line of a 1,000-paragraph English corpus sample at three widths against HarfBuzz, using exact ground truth that reshapes every candidate line.

- **Errors:** 4,812,111 of 5,560,372 lines (86.5%) were certified, and **0 were wrong**.
- **Coverage:** 371 faces certify at least 99% of their lines; 36 certify none.
- **Causes of failure** (78 faces under 99%, Figure 4b): 41 use AAT state machines the analyzer does not yet handle (reversed contextual, insertion, rearrangement, `kerx`/`kern` state machines); in 24 the font's substitutions genuinely touch common letters; in 9 the exhaustive island check's budget lowered the island bound below 4.

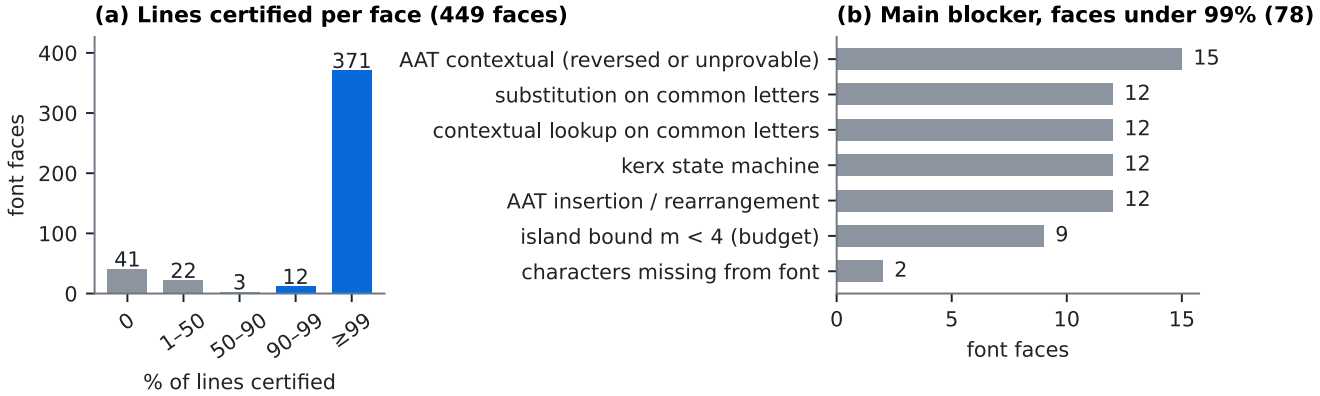


Figure 4. (a) Share of lines certified, per face. (b) The reason behind the most frequent unsafe character, for the faces under 99%.

5.2 · Today's shortcuts are not exact

The usual alternatives to full reshaping lose context, and the error is measurable. Against exact ground truth in HarfBuzz (14 fonts, 5,170 paragraphs, 6 widths; Zapfino is left out, because its long contextual swashes are exactly what the certificate hands back to the

shaper):

- **Shape once, then break** (what SkParagraph does) gets up to 1.2% of paragraphs wrong.
- **Per-word caches** (Minikin, Pretext) get up to 9.6% wrong.
- **The distilled model** gets 0.0% wrong.

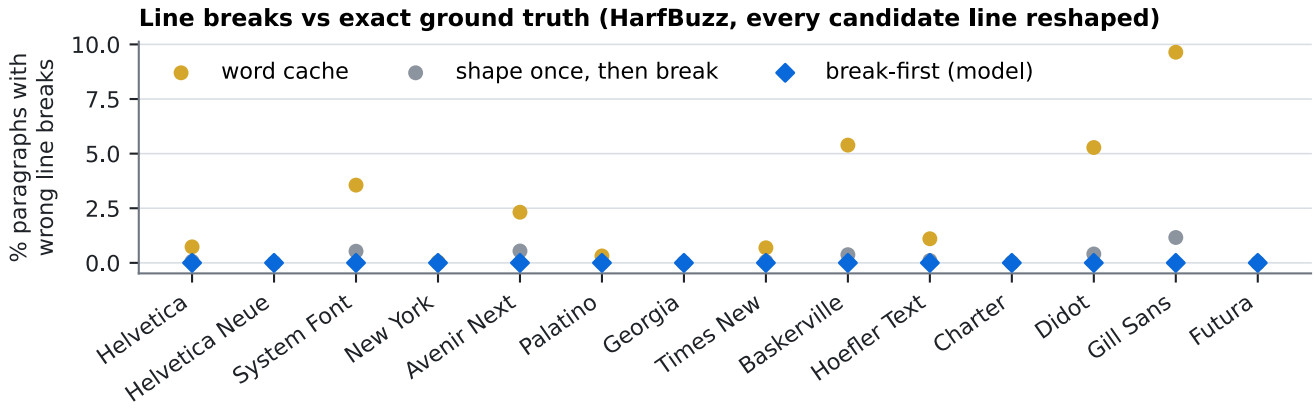


Figure 5. Share of paragraphs whose line breaks differ from exact ground truth (C prototype, HarfBuzz 15).

5.3 · Exactness inside Flutter

The real test is a production engine. A Flutter app (Flutter 3.47.5) loads the sidecar built on the host and compares every line break and every paragraph height with Flutter's own layout. The run covers nine corpora:

- English Python documentation, man pages and license texts;
- user-interface strings in English, German, French, Russian, Ukrainian and Greek, from macOS localizations.

Each paragraph is laid out at 6 font sizes (12, 13, 16, 18.4, 21.3, 24 px) × 12 box widths.

Platform	Paragraph layouts	Identical breaks	Identical heights	Deferred	Canary
macOS, release (all paragraphs)	2,547,000	2,524,902 / 2,524,902	2,524,902 / 2,524,902	0.87%	0.0 px
iOS 17.5 simulator, debug (300 per corpus)	194,184	192,193 / 192,193	192,193 / 192,193	1.03%	0.0 px
Android 15 emulator, release (300 per corpus)	194,184	192,176 / 192,176	192,176 / 192,176	1.03%	0.0 px

Table 2. One sidecar file, built once on macOS, checked against Flutter's layout on three platforms. Lines checked on macOS: 7,201,290.

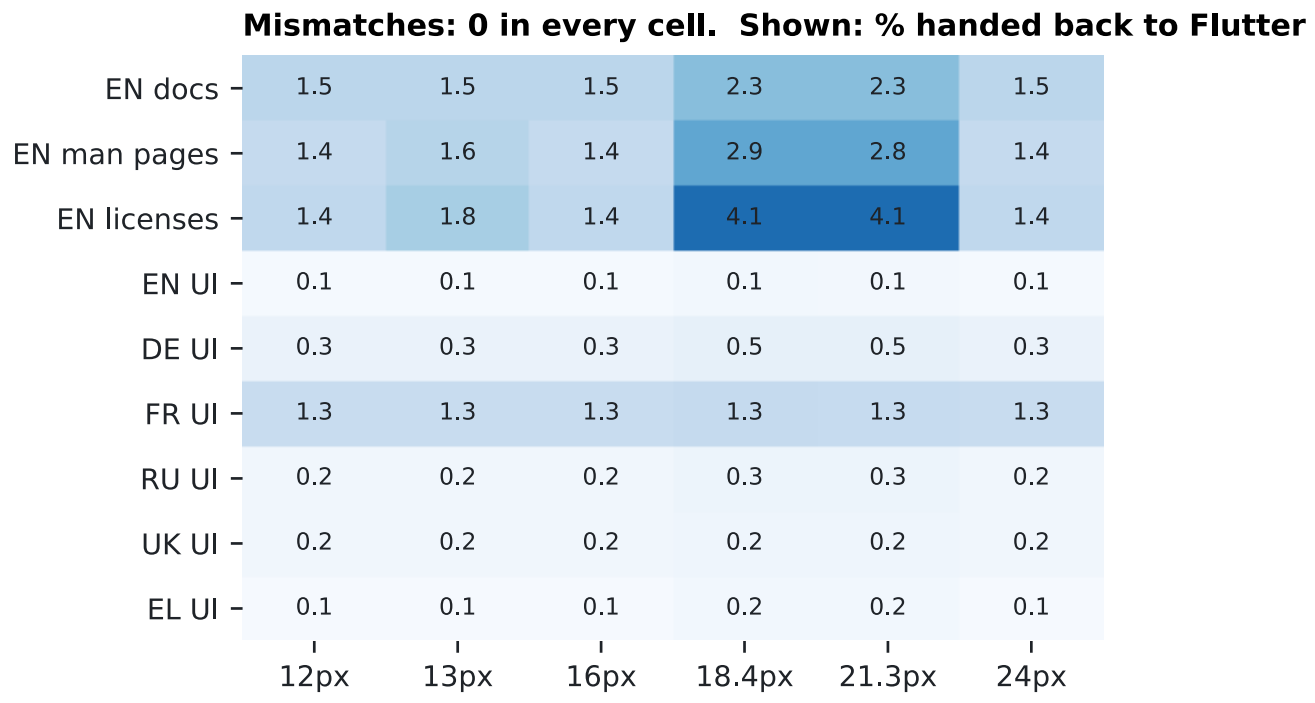


Figure 6. No cell has a single mismatch. Values are the share of paragraph layouts the breaker hands back to Flutter: words wider than the line, characters outside the model, and decisions it will not guess.

5.4 · Speed

All methods run on the same paragraphs, the ones break-first decides itself, at 16 px. Each time is the median of 5 passes inside one run of a macOS release build, and then the median of 3 such runs; the speedups below differ by at most 12% between runs.

- **Paragraph heights:** break-first is **18.5×** (docs) and **25.2×** (chat) faster than `TextPainter`.
- **Re-wrapping at a new width** is **11.6×** (docs) and **9.6×** (chat) faster than Flutter's re-layout, which already skips reshaping.
- **A warm word cache** is about as fast, but only after measuring every word once (the cold column), and it still breaks 3 docs and 3 chat paragraphs differently from Flutter.
- **End to end**, including paragraphs handed back to Flutter, the gain is 16.9× (docs) and 23.1× (chat).
- **Android emulator**, release, one run: 33.3× (docs) and 34.8× (chat).

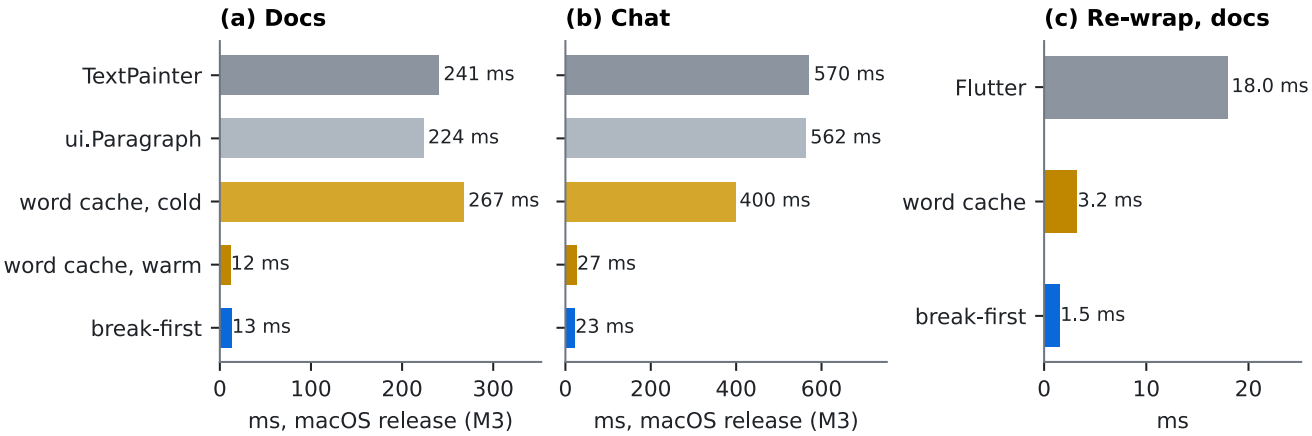


Figure 7. Time to compute every paragraph's height. (a) 5,089 Python documentation paragraphs at 400 px. (b) 23,736 UI messages at 280 px. (c) Re-wrapping the already laid-out docs paragraphs at a new width.

An app-level case: exact list extents. A chat list needs every item's height to place its scrollbar and jump to a message. For 24,000 messages, break-first computes all heights in 33 ms against 588 ms with `TextPainter` (18.0×), fallbacks included. 24,000 of 24,000 heights are equal, and the largest scroll-offset error over the whole list is 0.0 px. On the Android emulator the gain is 19.7×, again with 24,000 equal heights.

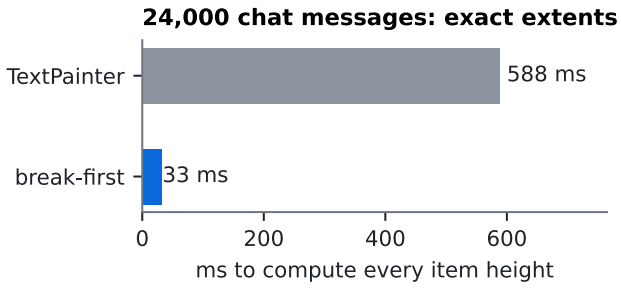


Figure 8. Computing every item's exact height for a chat list.

5.5 · Font sizes and float drift

Flutter's widths are linear in font size, so one file in font units serves every size. But SkParagraph accumulates positions across a paragraph in float32.

- **Exact sizes** (10, 12, 16, 20, 24, 32 px for Roboto): size/units-per-em is a short binary fraction and positions are exact.
- **Other sizes:** positions drift by up to 1.3e-05 of the x position, as much as 0.40 px on a paragraph-long line.

The breaker certifies exact sizes outright. At other sizes it defers any decision that a width error of 2 × 2.5e-05 of the x position could flip: drift at both ends of the line, each allowed 1.9× the largest drift measured. On Android the engine also truncates the font size to 1/64 px, which a single start-up probe detects. With both measures, all 6 sizes in Table 2 had zero mismatches on every platform.

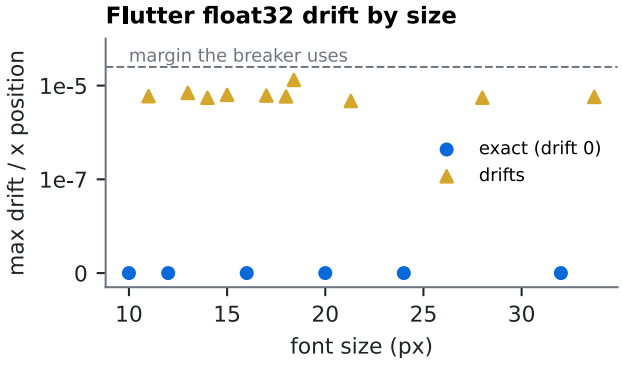


Figure 9. Largest drift between the model and Flutter, relative to x position, by font size.

6 · Related work

- **TeX's TFM lig/kern programs** are a hand-made per-font width automaton: the closest ancestor of the model, without contextual forms or a certificate.
- **PDF libraries** sum AFM widths and kerning pairs, ignoring substitution.
- **WebKit's simple text path** skips the shaper only when kerning and ligatures are off.
- **Android's Minikin and Pretext** measure and cache words, which loses context across words (§5.2).
- **HarfBuzz's** `UNSAFE_TO_CONCAT` makes reshaping after breaking cheaper, but keeps the shape-first order.
- `typeset` (2026) computes widths from advances and pair kerning with a HarfBuzz fallback, without contextual terms or a proof.

We found no prior static analysis of shaping tables. This survey is a web search, not an exhaustive review.

7 · Limitations

- **Scripts.** Latin, Greek and Cyrillic only; Arabic joining and Indic syllables are future work. Only plain, single-style, left-to-right paragraphs are covered.
- **Fonts.** The Flutter evaluation uses one font (Roboto); the census covers the faces installed on one Mac.
- **Proof.** The certificate relies on the trust base in §3, and the AAT analysis rejects reversed contextual subtables, rearrangement, insertion and `kerx` state machines rather than analysing them.
- **Measurement.** Mobile timings come from an emulator, not a phone.
- **Scope of the gain.** The speedup is for measurement and line breaking; painting is unchanged, and lines that are drawn are still shaped.

8 · Reproducibility

All code and results are in the accompanying folder:

- `certify.py`: the analyzer;
- `bfl.c`: the C prototype and checker;
- `census.py`: the census;
- `flutter_bench/`: the Flutter build step, breaker and harness;
- `paper/make_figures.py`: the figures and every number in this paper, read from the result files.

Environment: Flutter 3.47.5 (Dart 3.13.4), HarfBuzz 15.0.0, macOS on an Apple M3, iOS 17.5 simulator, and an Android API 35 emulator.